

A Comprehensive Guide to Git & GitHub

This guide consolidates information from the “GitHub Command Guide (Basic)” and the “git-uwu-guide (Senpai Edition)” to provide a thorough and friendly introduction to Git and GitHub.

Table of Contents

- 1. Initial Git Setup
 - 2. Starting a New Project
 - 3. Daily Git Workflow
 - 4. Syncing and Repository Fixes
 - 5. Advanced Git Operations
 - 6. Git Configuration & Tips
 - 7. GitHub Releases Guide
-

1. Initial Git Setup

Before you start using Git, you need to configure it with your identity. This information will be used to sign your commits.

Set Your Global Name and Email

These commands set your name and email for all repositories on your system.

```
# Hehe, let's get your name right, senpai! This command configures
# the author name to be used for all commits in the current user's
# account.
git config --global user.name "Your Name"

# And your email too, uwu~ This command configures the author email
# to be used for all commits in the current user's account.
git config --global user.email "your.email@example.com"
```

Pro-Tip: If you want to use a different name/email for a specific project, you can run these commands without the `--global` flag inside that project's directory.

Verify Your Configuration

To check your current Git configuration, use the following command:

```
# Let's see what you've got! This command displays all the
# configuration settings for the current repository and the global
# configuration.
git config --list
```

2. Starting a New Project

This section covers how to initialize a new local repository and push it to GitHub for the first time.

Initialize a New Repository

```
# Let the adventure begin! Initializes an empty Git repository in
# the current directory by creating a new .git subdirectory.
git init
```

Add and Commit Your Files

```
# Get in the bag! Stages all new and modified files in the current
# directory and subdirectories for the next commit.
git add .
```

```
# Seal the deal with a message, uwu~ Records a snapshot of the
# staged changes to the repository's history. The -m flag allows for
# an inline commit message.
git commit -m "Initial commit"
```

Link to a GitHub Repository and Push

```
# Connect to the mothership! Creates a new connection to a remote
# repository. 'origin' is the conventional name for the primary remote
# repository.
git remote add origin git@github.com:your-username/your-repo.git
```

```
# Blast off! Pushes the 'main' branch to the 'origin' remote. The -u
# flag sets the remote branch as the upstream reference for the local
# branch, allowing for 'git pull' and 'git push' without arguments in
# the future.
git push -u origin main
```

Note: Replace `your-username` and `your-repo.git` with your actual GitHub username and repository name. Also, your primary branch might be named `master` instead of `main`.

3. Daily Git Workflow

This is the typical sequence of commands you'll use in your day-to-day development.

1. Fetch and Rebase

Before you start working, it's good practice to get the latest changes from the remote repository.

```
# What's new, pussycat? Downloads commits, files, and refs from a  
# remote repository into the local repo, but does not merge them.  
git fetch origin
```

```
# Let's get you up to date! Fetches changes from the remote and  
# reapplies local commits on top of the remote changes. This avoids a  
# merge commit and maintains a linear history.  
git pull --rebase origin main
```

2. Add and Commit Your Changes

As you work, you'll make changes. When you're ready to save a snapshot of your work, you'll add and commit.

```
# Stage your changes  
git add .
```

```
# Commit with a message  
git commit -m "feat: Add new feature"
```

3. Push Your Changes

```
# Ship it! Uploads local branch commits to the remote repository.  
git push origin main
```

4. Syncing and Repository Fixes

Sometimes, your local repository can get out of sync with the remote. Here's how to fix common issues.

Check Your Repository Status

```
# How are we looking? Displays the state of the working directory  
# and the staging area.  
git status
```

```
# Where are we going? Lists the remote repositories, with their  
# URLs.  
git remote -v
```

Common Syncing Scenarios

- If your local branch is ahead of the remote:

```
# Your local branch has commits that the remote does not. Push them.  
git push origin main
```

- **If your local branch is behind the remote:**

```
# The remote has commits that your local branch does not. Pull them.  
git pull origin main
```

- **If your local and remote branches have diverged:** This means both have unique commits. Use rebase to apply your local commits on top of the remote changes.

```
git pull --rebase origin main
```

Drastic Measures (Use with Caution!)

Warning: Reset your local branch to match the remote exactly. This will discard all of your local changes and commits that have not been pushed.

```
# Nuke it from orbit! Resets the current  
# branch head to the specified commit and updates the working  
# directory to match. All local changes will be lost.  
git fetch origin  
git reset --hard origin/main
```

Warning: Force push your local changes to overwrite the remote. This will overwrite the history on the remote repository. Only do this if you are absolutely sure you know what you are doing.

```
# Don't do this unless you're sure, senpai! Forces the push,  
# overwriting the remote branch with the local one. This can  
# discard commits on the remote.  
git push origin main --force
```

What to do if you accidentally delete the `.git` directory

Oh no! If you've deleted the `.git` directory, your project is just a folder of files. But don't worry, senpai, we can fix it! Here are a couple of ways to resync your local project with the remote repository on GitHub.

Option 1: Fresh Re-initialize (Overwrites Remote History)

Warning: This is a destructive action!

Use this option if you only care about your current local files and you're okay with overwriting the project's history on the remote.

```
# Go to your project directory
```

```
cd /path/to/your/project

# Start fresh by reinitializing Git
git init

# Ensure the local branch is named 'main'
git branch -M main

# Connect to the remote repository
git remote add origin https://github.com/USERNAME/REPO.git

# Stage all your files for commit
git add .

# Make a fresh initial commit
git commit -m "Reinitialize repository"

# Force push to overwrite the remote with your local state
git push -u origin main --force
```

Option 2: Restore Remote History (Preserves History)

Use this safer option if you want to keep the remote repository's history and just add your local changes on top of it.

1. Clone the repository to a new folder:

```
# Clone into a new directory to avoid conflicts.
git clone https://github.com/USERNAME/REPO.git my_project_clone
```

2. Copy your local changes into the new clone: Use `rsync` to copy your work into the new clone. This will overwrite the clone's files with your latest work, without touching the `.git` directory.

```
# Make sure to exclude the .git directory from the source!
rsync -av --progress --exclude '.git' /path/to/your/original/project/ my_project_clone/
```

3. Commit and push your changes: Now, go into the clone, and commit your changes. This will add your work as a new commit on top of the existing history.

```
cd my_project_clone
git add .
git commit -m "Resync local changes"
git push
```

A note on branches (master vs. main): When you re-initialize, Git might create a `master` branch by default. If you omit `git branch -M main`, your local branch will be named `master`. If your remote uses `main`, you can create and switch to it like this: `git checkout -b main`. You can then set `main` as the default branch in your repository's settings on GitHub.

5. Advanced Git Operations

Here are some more advanced commands that can be very useful.

Viewing Commit History

```
# Look at this beautiful history! A powerful log command that shows  
# a condensed, graphical representation of the commit history for all  
# branches.  
git log --oneline --graph --decorate --all
```

Undoing Commits

```
# Oops, I didn't mean to do that! Undoes the last commit, but keeps  
# the changes in the staging area.  
git reset --soft HEAD~1
```

```
# I really didn't mean to do that! Undoes the last commit and  
# discards all changes from that commit.  
git reset --hard HEAD~1
```

Managing Branches

```
# Time for a side quest! Creates a new branch and switches to it.  
git checkout -b new-feature-branch
```

```
# Tidy up! Tidy up! Removes untracked files from the working  
# directory. -f is for force, -d is for directories.  
git clean -fd
```

6. Git Configuration & Tips

.gitignore

A `.gitignore` file tells Git which files and folders to ignore. This is useful for preventing secrets, logs, and build artifacts from being committed.

- **Example .gitignore entries:** `node_modules/`, `*.log`, `.env`, `__pycache__/`, `*.swp`
- **Generator:** You can generate a `.gitignore` file for your project at toptal.com/developers/gitignore.

Handling Large Files (HTTP Post Buffer)

If you have trouble pushing large files or have a large commit history, you might get a timeout error. You can increase Git's HTTP buffer to help with this.

```
# MOAR POWER! Set the HTTP post buffer to 500MB (524288000 bytes).  
# This can help with pushing large files.  
git config --global http.postBuffer 524288000
```

```
# For even bigger files, you can set it to 1GB (1048576000 bytes)!  
git config --global http.postBuffer 1048576000
```

Switching Between SSH and HTTPS Remotes

You can switch the remote URL between HTTPS and SSH protocols.

```
# Let's try a different route. Changes the URL of the remote
# repository.
git remote set-url origin git@github.com:your-username/your-repo.git
```

Creating a Git Alias

You can create shortcuts for your favorite commands. A special move! Creates a global alias for a sequence of commands. The `!` at the beginning allows the use of shell commands.

```
# The general syntax for creating an alias is:
git config --global alias.<alias name> '<commands to exec>'
```

```
# Here is our "pushnyaa" alias:
git config --global alias.pushnyaa '!git push && echo "push complete senpai~"'
```

Now you can just run `git pushnyaa` to push your changes.

7. GitHub Releases Guide

GitHub releases are a way to package and distribute specific versions of your project. They are built on top of Git tags.

1. Tagging a Commit

A release always points to a specific commit. Tags can be lightweight or annotated.

- **Lightweight tag:** `bash git tag v1.0.0`
- **Annotated tag (recommended):** `bash git tag -a v1.0.0 -m "First stable release"`
- **Push tags to GitHub:** `bash git push origin v1.0.0 # Or push all tags git push origin --tags`

2. Creating a Release on GitHub

1. Go to your repository on GitHub.
2. Click on `Releases` → `Draft a new release`.
3. Select a tag (existing or new).
4. Add:
 - Release title
 - Description/Changelog (Markdown supported)
5. Optionally, attach binary assets.
6. Click `Publish release`.

GitHub shows your release as a downloadable package tied to the tag.

3. Automating Releases

- **Using GitHub CLI:** `bash gh release create v1.0.0 ./my-binary.zip --title "v1.0.0" --notes "First release"`
- **Using GitHub Actions:** Automatically build binaries and create releases on pushes or tags.

4. Key Notes

- Tags are the foundation; releases are enhanced tags.
- Releases allow easy download of binaries without cloning the repo.
- Follow semantic versioning (v1.0.0, v1.0.1, etc.).